

Maestro Agrícola: uma Interface Hands-Free e Fail-Safe para Comando de Robôs Agrícolas com Visão, Voz e IA Local

Átila Capozzoli Ribeiro Rodrigues, Felipe Gonçalves Vidal e Rafael José de Souza Marques
Equipe AgroTurtles
Programa AI Glasses Brasil 2026, Brasil

Resumo—Este artigo apresenta o Maestro Agrícola, uma interface hands-free para solicitar ações a robôs agrícolas por alvo visual, voz e confirmação explícita. O sistema separa a linguagem natural do caminho de controle: um aplicativo Android/Kotlin obtém uma foto pela pilha Meta Wearables Device Access Toolkit (DAT), resolve um QR previamente mapeado, classifica localmente uma intenção restrita e somente então cria um comando JSON expirável e idempotente. Um bridge valida o comando antes de encaminhá-lo ao ROS 2 e ao Nav2 em simulação. Um modelo Qwen2.5 local é mantido apenas como assistente conversacional, sem acesso ao robô. Na etapa pré-hardware, o fluxo `datDebug` com DAT 0.9.0 e `MockDeviceKit` foi executado em Android físico até o TurtleBot 4 no Gazebo. A avaliação controlada do classificador obteve 64/64 acertos e zero aceites perigosos no corpus original; uma regressão adicional obteve 48/48 nos seis rótulos atuais. Testes de conflito visual-falado, cancelamento, expiração e estado inválido resultaram em falha segura. Os resultados demonstram viabilidade pré-hardware, mas não comprovam câmera ou áudio nos Meta Wearables físicos, desempenho agrônômico ou redução de custo em campo.

Index Terms—agricultura de precisão, interação humano-robô, computação vestível, robótica móvel, processamento local, segurança operacional

I. INTRODUÇÃO

Máquinas agrícolas e robôs móveis podem delegar navegação e controle de baixo nível à autonomia embarcada, mas a solicitação humana de uma nova tarefa ainda pode depender de telas, menus ou coordenadas. Sob sol, poeira, ruído e uso de luvas, essa interface acrescenta atrito e desloca a atenção do operador. O Maestro Agrícola investiga uma alternativa em três atos: olhar para um alvo permitido, falar uma intenção restrita e confirmar a interpretação antes de qualquer movimento.

O trabalho não propõe substituir navegação, desvio de obstáculos ou segurança funcional do veículo. Essas responsabilidades permanecem no robô. A contribuição é uma fronteira de interação verificável entre linguagem humana e uma máquina física, com quatro propriedades: (i) alvo pertencente a uma lista permitida; (ii) intenção operacional local e restrita; (iii) confirmação explícita; e (iv) validação de contrato e estado no lado robótico.

O protótipo integra Android/Kotlin, DAT 0.9.0, reconhecimento e síntese de fala nativos, leitura QR local, WebSocket, ROS 2, Nav2 e Gazebo. O DAT fornece sessão e câmera para experiências hands-free e inclui `MockDeviceKit` para desenvolvimento sem óculos físicos [1]. ROS 2 e Nav2

fornece, respectivamente, a infraestrutura distribuída e a navegação móvel usadas na bancada simulada [2], [3].

II. REQUISITOS E MODELO DE SEGURANÇA

A. Escopo operacional

O contrato 1.0 aceita apenas SPRAY, DOCK e UNDOCK. SPRAY exige um alvo do tipo `MAPPED_PLOT`; DOCK e UNDOCK rejeitam alvo. Todo comando inclui `UUID`, instante de criação, expiração entre 1 e 30.000 ms e `confirmed=true`. O bridge rejeita schema inválido, comando vencido, repetido ou incompatível com o estado corrente.

No lado da interação, `CONFIRM` e `CANCEL` controlam uma operação pendente; `UNKNOWN` nunca produz movimento. Divergência entre QR e ID falado limpa o alvo resolvido. Assim, a ausência de evidência suficiente termina em pedido de repetição ou cancelamento, não em uma coordenada padrão.

B. Separação entre IA e autoridade

O `LocalIntentClassifier` é a única autoridade linguística operacional. Ele combina regras determinísticas com um classificador linear softmax e limiar 0,40. O artefato canônico possui 213 exemplos de treino, seis rótulos e 729.056 bytes. A execução ocorre localmente no Android, sem servidor de inferência.

O Qwen2.5-1.5B [4] é executado opcionalmente por `llama.cpp` [5]. O `LanguageRouter` encaminha ao assistente somente textos já classificados como `UNKNOWN`; uma gramática limita a saída a `CHAT` ou `OUT_OF_SCOPE`. O componente não recebe tipos de comando, alvo, WebSocket ou ROS. Essa separação foi adotada porque o benchmark operacional do Qwen registrou três aceites perigosos em 48 casos.

III. ARQUITETURA E IMPLEMENTAÇÃO

A. Entrada visual e áudio

No flavor `dat`, o `PlatformFrameSource` implementa permissão, registro, seleção de dispositivo, `DeviceSession`, `addCamera()`, captura sob demanda, timeout, cancelamento e liberação de recursos conforme o ciclo do DAT. A foto é entregue como `PhotoData.Bitmap` ou HEIC e processada em memória pelo ZXing [6]. O decodificador limita a maior dimensão a 1.600 pixels, tenta rotações de 90, 180 e 270 graus e rejeita zero ou múltiplos candidatos.

Na etapa pré-hardware, o MockDeviceKit simula um Ray-Ban Meta e um provider interno, não exportado, transmite a fixture `plot-03.png` por pipe. O modo é opt-in; sem a propriedade de build correspondente, não há fallback silencioso para o dispositivo simulado.

Voz e TTS usam `SpeechRecognizer` e `TextToSpeech` do Android [7], [8]. O app solicita preferência offline, mas o provedor efetivo pode variar por aparelho. O código atual usa a rota de áudio ativa e ainda não seleciona nem comprova explicitamente a rota Bluetooth dos óculos.

B. Aplicativo, contrato e robótica

O fluxo principal é:

```
DAT/QR → TargetResolver
→ LocalIntentClassifier →
InteractionEngine → confirmação →
JSON/WebSocket → bridge → ROS 2/Nav2/Gazebo.
```

O `InteractionEngine` mantém os estados de alvo, intenção e confirmação. Somente a transição de confirmação válida cria um comando. O bridge mantém cache de até 1.024 UUIDs para idempotência, associa `plot-01`, `plot-02` e `plot-03` a poses permitidas e usa ações explícitas para docking e undocking. A conclusão de `SPRAY` deixa o robô no alvo; não existe retorno automático à doca.

O JSON versionado atua como fronteira entre o aplicativo e a robótica. A validação estrutural segue o princípio de schemas declarativos do JSON Schema [9], complementada por regras de expiração, estado e deduplicação.

IV. METODOLOGIA DE AVALIAÇÃO

A avaliação foi organizada em quatro níveis.

- 1) **IA operacional:** corpus original com 64 frases das quatro classes iniciais; corpus de campo com 48 frases balanceadas entre os seis rótulos atuais; limiar 0,40 e contagem de aceites perigosos.
- 2) **Regressão de software:** 65 testes portáteis versionados e 36 testes do bridge, incluindo contrato, expiração, idempotência, alvo e estados.
- 3) **E2E pré-hardware:** Android físico com `datDebug`, DAT 0.9.0 e MockDeviceKit, conectado por `WebSocket` ao ROS 2/Nav2/Gazebo.
- 4) **Falhas seguras:** `SPRAY` dockado, conflito entre QR e voz, cancelamento durante confirmação e entrada desconhecida.

Também foi analisado o Qwen como hipótese de classificador operacional. Esse ensaio usa os mesmos seis rótulos, mas serve para decidir sua fronteira de autoridade, não para comparar modelos de linguagem de forma geral.

V. RESULTADOS

O ciclo E2E repetiu a captura de `plot-03`, realizou undock confirmado, navegou até o alvo, permaneceu no ponto, retornou à doca somente após `DOCK` e executou novo `UNDOCK`. `SPRAY` enquanto dockado foi recusado sem saída implícita da doca. O conflito `visual=plot-03` versus `voz=plot-01` terminou

Tabela I
EVIDÊNCIAS CONTROLADAS NO SNAPSHOT AUDITADO

Evidência	Resultado
Corpus original	64/64; macro-F1 1,00; zero aceite perigoso
Corpus de seis rótulos	48/48 na regressão reproduzível
Qwen como controle	36/48; macro-F1 0,7384; 3 aceites perigosos
Testes de software E2E pré-hardware	65 portáteis versionados; 36 do bridge <code>UNDOCK</code> → <code>SPRAY plot-03</code> → <code>DOCK</code> → <code>UNDOCK</code>
Falhas seguras	estado inválido, conflito e cancelamento sem movimento

em ambiguidade; `CANCEL` durante a confirmação não criou comando.

Uma medição física histórica registrou 540 inferências no Motorola Edge 40 Neo, com mediana de 229 μ s, p95 de 1.013 μ s e zero divergências. Contudo, os hashes mostram que o benchmark pertence a modelo e APK anteriores ao candidato atual. Esses números são mantidos como referência histórica e não como desempenho congelado da entrega corrente.

VI. DISCUSSÃO, PRIVACIDADE E LIMITAÇÕES

Os resultados sustentam a viabilidade da arquitetura pré-hardware e a eficácia dos guardrails nos casos versionados. Eles não medem generalização de fala espontânea, ruído agrícola, robustez de QR em poeira, segurança funcional, consumo de bateria com todos os componentes simultâneos ou desempenho em maquinário real.

O código do Maestro não cria arquivos de foto, áudio ou transcrição. O frame é processado em memória, o backup Android está desabilitado e os opt-outs de analytics e crash reporting do DAT estão declarados. Ainda assim, “não persistir pelo Maestro” não prova o comportamento interno do Android, do provedor de fala ou do SDK. Uma auditoria física com Meta Wearables reais continua necessária.

A faixa de 20–30% exibida no pitch é uma oportunidade de negócio a validar em piloto controlado; não deriva dos experimentos apresentados e, portanto, não é reportada como impacto obtido. O estudo também não compara a interface hands-free com tablet ou notebook por tempo, erro ou carga cognitiva.

VII. PRÓXIMAS ETAPAS

Os próximos gates são: (i) substituir o MockDeviceKit por Meta Wearables físicos e registrar firmware, pareamento, câmera e rota de áudio; (ii) repetir benchmark, bateria, temperatura e auditoria de persistência no artefato final; (iii) integrar um robô físico em ambiente controlado; e (iv) conduzir piloto com métricas de tempo, retrabalho, aceites perigosos e custo operacional. Cada gate preservará a meta de zero aceite perigoso antes de ampliar intenções ou autonomia.

VIII. CONCLUSÃO

O Maestro Agrícola demonstra que uma experiência “olhar, falar e confirmar” pode ser estruturada como uma fronteira segura e testável entre linguagem natural e robótica. A separação entre classificador operacional, assistente conversacional e autonomia do robô reduz a superfície de falha: o LLM conversa, o classificador decide dentro de um vocabulário restrito, o humano confirma e o bridge valida antes do ROS 2. O protótipo fecha o ciclo em pré-hardware com DAT 0.9.0 e MockDeviceKit; a validação nos Meta Wearables e no campo permanece explicitamente aberta.

AGRADECIMENTOS

A equipe agradece ao Programa AI Glasses Brasil e às instituições Meta, CEIA e AKCIT pelo contexto de desenvolvimento e avaliação do projeto.

REFERÊNCIAS

- [1] Meta, “Meta Wearables Device Access Toolkit for Android,” GitHub repository, 2026. [Online]. Available: <https://github.com/facebook/meta-wearables-dat-android>. Acesso em: 22 ago. 2026.
- [2] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, Art. no. eabm6074, 2022. doi: 10.1126/scirobotics.abm6074.
- [3] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, “The Marathon 2: A Navigation System,” arXiv:2003.00368, 2020. [Online]. Available: <https://arxiv.org/abs/2003.00368>.
- [4] A. Yang *et al.*, “Qwen2.5 Technical Report,” arXiv:2412.15115, 2024. [Online]. Available: <https://arxiv.org/abs/2412.15115>.
- [5] ggml-org, “llama.cpp,” GitHub repository, 2026. [Online]. Available: <https://github.com/ggml-org/llama.cpp>. Acesso em: 22 ago. 2026.
- [6] ZXing Project, “ZXing: Zebra Crossing barcode scanning library for Java and Android,” GitHub repository, 2026. [Online]. Available: <https://github.com/zxing/zxing>. Acesso em: 22 ago. 2026.
- [7] Google, “SpeechRecognizer,” *Android Developers*. [Online]. Available: <https://developer.android.com/reference/android/speech/SpeechRecognizer>. Acesso em: 22 ago. 2026.
- [8] Google, “TextToSpeech,” *Android Developers*. [Online]. Available: <https://developer.android.com/reference/android/speech/tts/TextToSpeech>. Acesso em: 22 ago. 2026.
- [9] JSON Schema, “JSON Schema Draft 2020-12.” [Online]. Available: <https://json-schema.org/draft/2020-12>. Acesso em: 22 ago. 2026.
- [10] AGCO Corporation, “AGCO Improved Productivity and Accuracy Using Assisted Reality with Glass,” 18 jul. 2017. [Online]. Available: <https://investors.agcocorp.com/news-releases/news-release-details/agco-improved-productivity-and-accuracy-using-assisted-reality>. Acesso em: 22 ago. 2026.
- [11] F. G. Vidal, R. J. S. Marques, and Á. C. R. Rodrigues, “Maestro Agrícola,” GitHub repository, 2026. [Online]. Available: <https://github.com/vvvvdal/maestro-agricola>. Acesso em: 22 ago. 2026.